

Reviewer Pack — Minimal Rank of Algebraic K-Theory and DPLL Proof Depth Corr...

Entry c13b199e6edb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 21:49:23 UTC

Minimal Rank of Algebraic K-Theory and DPLL Proof Depth Correlation

Entry ID: c13b199e6edb **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 21:49:23 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-Theory **Field B** (complexity object): DPLL Search Trees

Statement:

For every CNF φ with n variables, the minimal rank of its associated algebraic K-group ($\text{rank}_k(\varphi)$) is linearly correlated with its DPLL proof depth $d(\varphi)$, such that $\text{rank}_k(\varphi) = \Theta(d(\varphi))$. Furthermore, for all instances with property $P(\varphi)$, the following holds: if $d(\varphi) \leq c \cdot n^2$ for a constant $c > 0$, then $\text{rank}_k(\varphi) \geq \min\{r : r \leq c \cdot n^2\}$.

Rationale (proposer's reasoning):

Algebraic K-theory provides a rich framework for understanding and classifying algebraic structures, including those that arise in complexity theory. The minimal rank of an algebraic K-group is a measure of its complexity. If this invariant correlates with DPLL proof depth, it could offer insights into the structure of SAT instances and their resolution complexities.

Taxonomy category: Algebraic_K_Theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 301a2756108959eb

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given CNF φ with n variables, if the correlation coefficient between $\text{rank}_k(\varphi)$ and $d(\varphi)$ is ≥ 0.7 , and for all φ with property $P(\varphi)$, $\text{rank}_k(\varphi)$ meets the inequality $r \leq c \cdot n^2$ where $c > 0$ is a constant.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(10 * n):
            clause = [random.randint(-n, -1), random.randint(1, n)]
            clauses.append(clause)
        return clauses

    def dpll(cnf, assignment={}):
        if not cnf:
            return True
        literal = next((l for l in range(-n, n+1) if l not in assignment and -l not in assignment))
        if literal is None:
            return False

        def propagate(lit):
            new_cnf = []
            for clause in cnf:
                if lit in clause:
                    continue
                elif -lit in clause:
                    clause.remove(-lit)
                    if not clause:
                        return False
                else:
                    new_cnf.append(clause)
            return new_cnf

        assignment[literal] = True
        if propagate(literal):
            if dpll(new_cnf, assignment):
```

```

        return True
    del assignment[literal]

    assignment[-literal] = True
    if propagate(-literal):
        if dpll(new_cnf, assignment):
            return True
    del assignment[-literal]

    return False

def algebraic_k_group(cnf):
    # Placeholder implementation for minimal rank calculation
    # This is a dummy function and should be replaced with actual computation
    return random.randint(1, n)

def property_P(cnf):
    # Placeholder implementation for property P
    # This is a dummy function and should be replaced with actual check
    return True

n = 30
cnf = generate_cnf(n)
rank_k = algebraic_k_group(cnf)
dpll_depth = dpll(cnf)

if dpll_depth is None:
    dpll_depth = float('inf')

correlation_coefficient = (rank_k - dpll_depth) / (n * n)

return {
    "metric_name": "Correlation Coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": abs(correlation_coefficient) >= 0.7 and property_P(cnf),
    "counterexample": "" if property_P(cnf) else "property_P_failed"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:

```

```

    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    counterexample = "property_P_failed"
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_115ac522.py", line 96, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_115ac522.py", line 75, in run_trial
    dpll_depth = dpll(cnf)
                  ^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_115ac522.py", line 50, in dpll
    if dpll(new_cnf, assignment):
           ^^^^^^^
NameError: name 'new_cnf' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means we cannot verify the conjecture's conditions. | next: Re-run the test ensuring it completes without crashing and produces results to validate or falsify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	25974
2	preregistration	ollama_remote	glm4:latest	0	0	14247
3	novelty	ollama_remote	glm4:latest	0	0	8474
4	novelty	ollama_remote	glm4:latest	0	0	8746
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18664
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19712
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14537
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9904

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	judge	ollama_remote	glm4:latest	0	0	12209

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 132467 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/c13b199e6edb.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/c13b199e6edb.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/c13b199e6edb.tar.gz (if generated)